

2. Unrolled List

–Delete Element

–Program Analysis

Program:

```
void deleteElement(int val) {
    if (head == NULL) {
        cout << "The list is empty. Nothing to delete.\n";
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {

                // Element found: Shift remaining elements left
                for (int j = i; j < temp->numElements - 1; j++) {
                    temp->elements[j] = temp->elements[j + 1];
                }
                temp->numElements--;

                cout << "Element " << val << " deleted
successfully.\n";

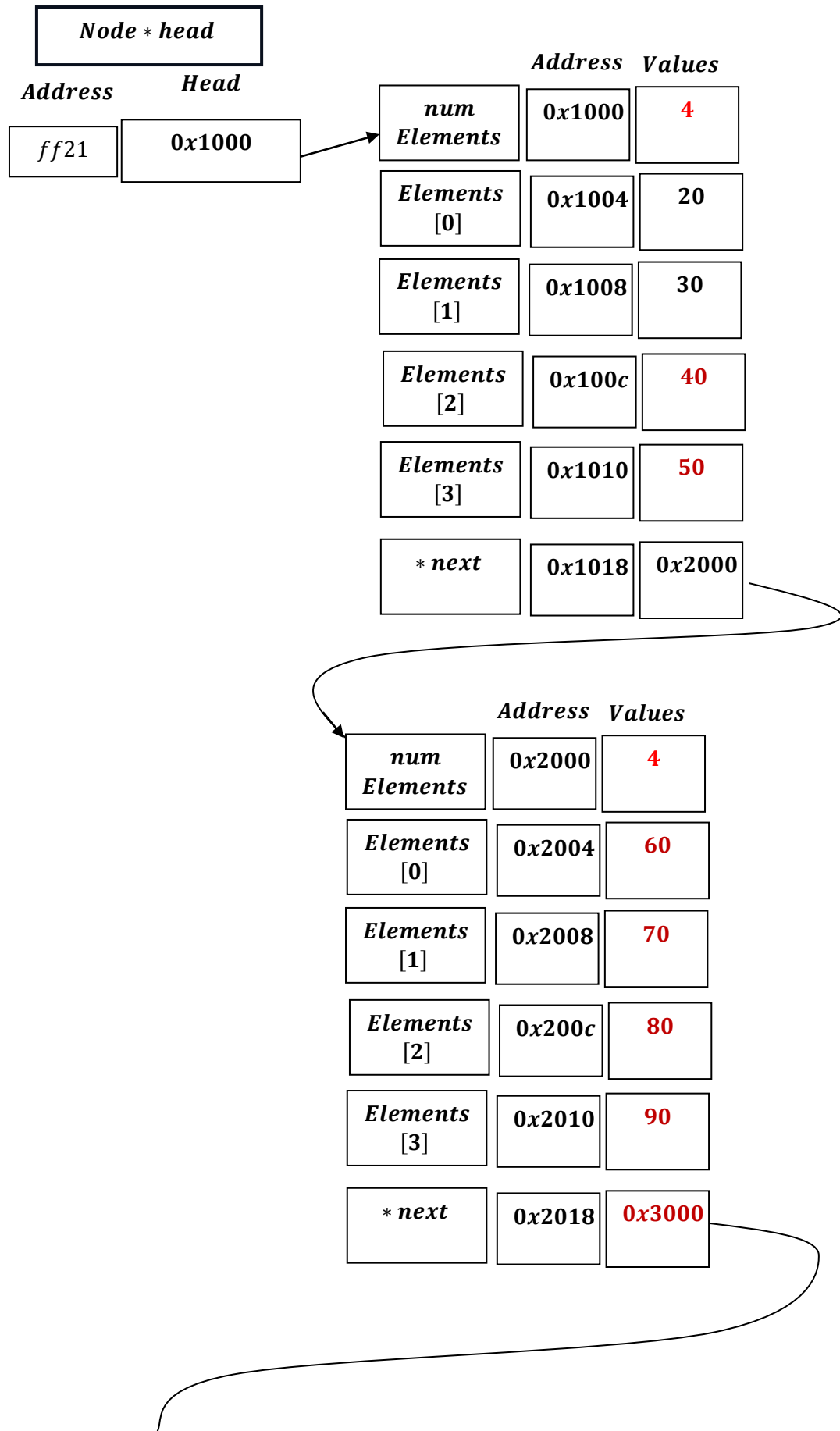
                // Check if the node is now completely empty
                if (temp->numElements == 0) {
                    if (prev == NULL) {
                        head = temp->next;
                        free(temp);
                    } else {
                        prev->next = temp->next;
                        free(temp);
                    }
                    cout << "An empty node was removed from the
list.\n";
                }
            }
        }
    }
}
```

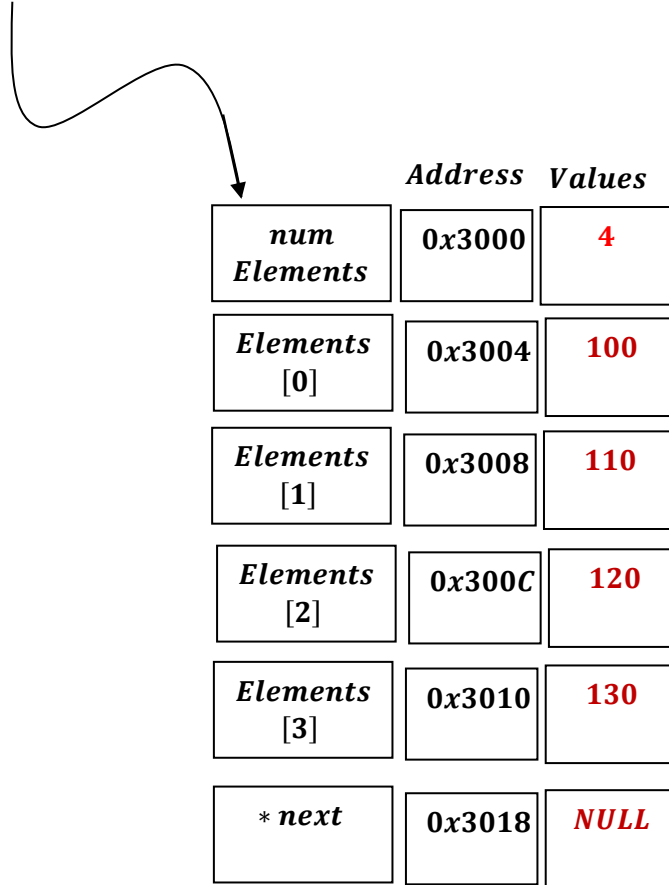
```
        return;
    }
}
prev = temp;
temp = temp->next;
}

cout << "Element " << val << " not found in the list.\n";
}
....

    case 2:
        cout << "Enter integer value to delete: ";
        cin >> value;
        deleteElement(value);
        break;
```

Program Analysis





	Address	Values
num <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
<i>* next</i>	0x3018	NULL

Function call:

```
case 2:
    cout << "Enter integer value to delete: ";
    cin >> value;
    deleteElement(value);
    break;
```

Say, value is : 110

```
void deleteElement(int val)
```

deleteElement(val: 110)

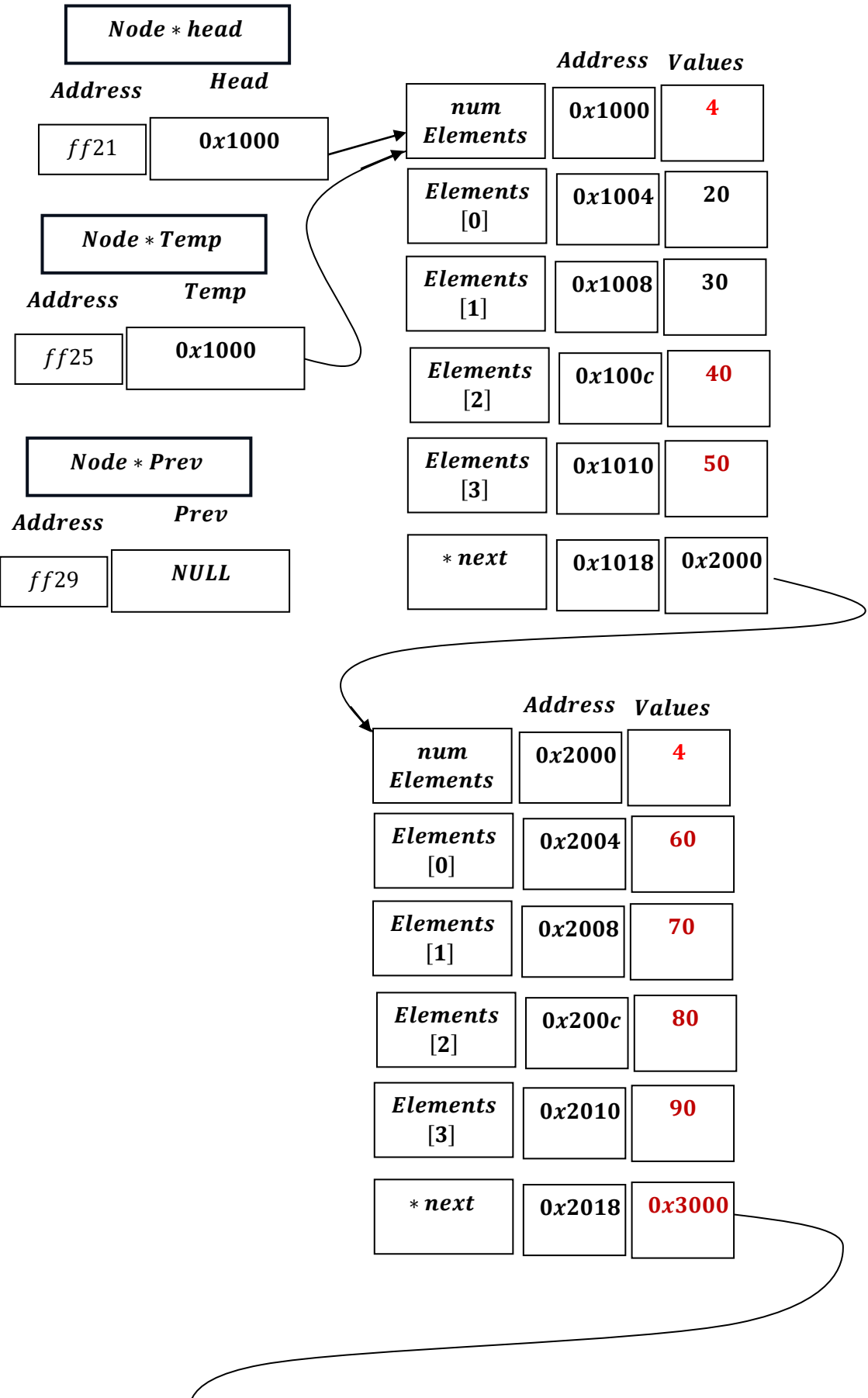
```
if (head == NULL) {
    cout << "The list is empty. Nothing to delete.\n";
    return;
}
```

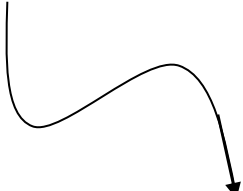
if (head = 0x1000 ≠ NULL) [Hence condition is False.]

```
struct Node* temp = head;
```

And,

```
struct Node* prev = NULL;
```





	Address	Values
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL

```

while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {

            // Element found: Shift remaining elements left
            for (int j = i; j < temp->numElements - 1; j++) {
                temp->elements[j] = temp->elements[j + 1];
            }
            temp->numElements--;

            cout << "Element " << val << " deleted
successfully.\n";

            // Check if the node is now completely empty
            if (temp->numElements == 0) {
                if (prev == NULL) {
                    head = temp->next;
                    free(temp);
                } else {
                    prev->next = temp->next;
                    free(temp);
                }
                cout << "An empty node was removed from the
list.\n";

```

```

    }
    return;
}
}
prev = temp;
temp = temp->next;
}

```

$while(Temp = 0x1000 \neq NULL) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$for\ i = 0; i = 0 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$if(temp \rightarrow Elements[0]: 20 \neq 110) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 0 + 1 = 1.$

$for\ i = 1; i = 1 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$if(temp \rightarrow Elements[1]: 30 \neq 110) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 1 + 1 = 2.$

for $i = 2; i = 2 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[2]: 40 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 2 + 1 = 3.$

for $i = 3; i = 3 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[3]: 50 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

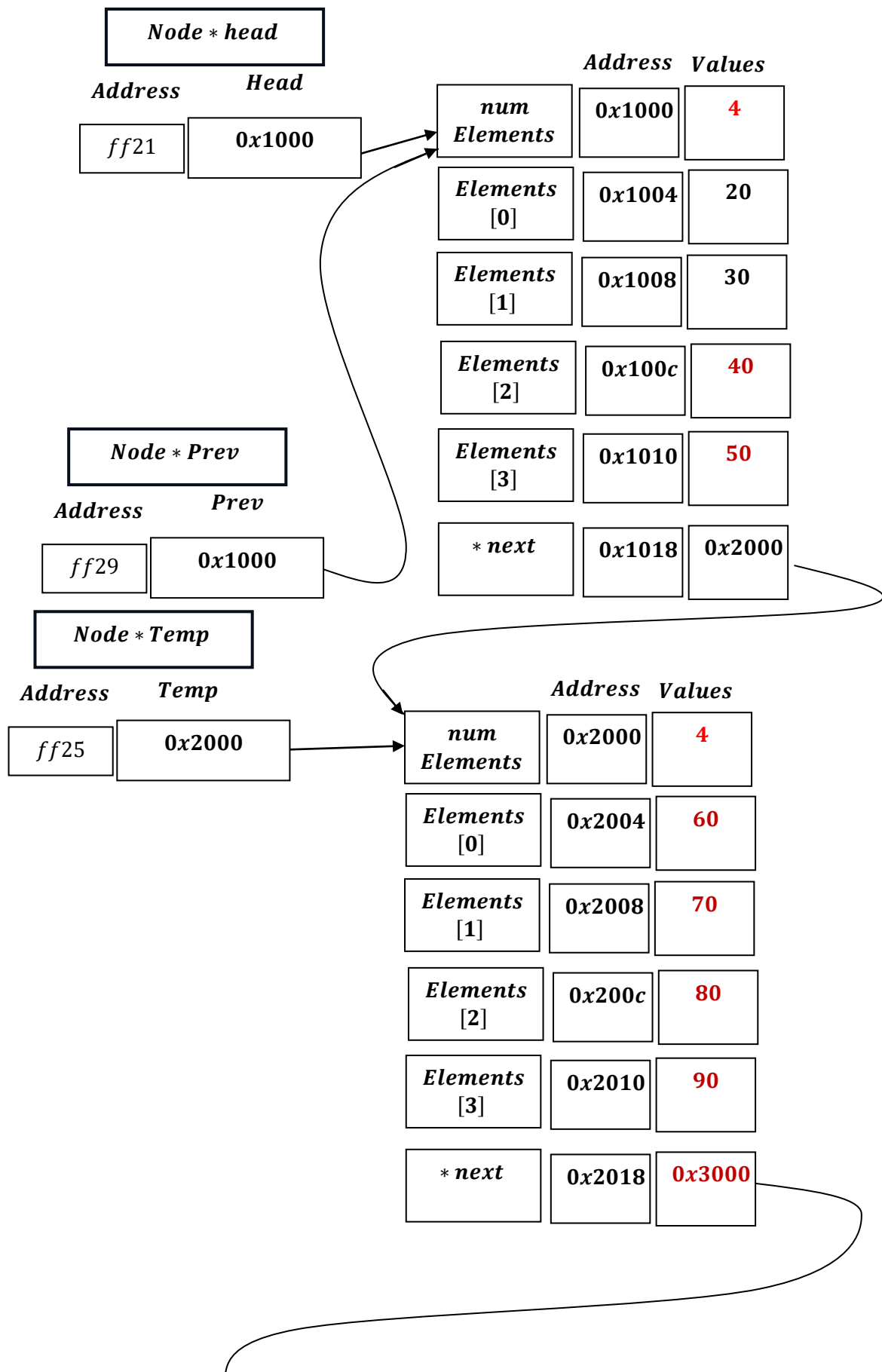
$i = i + 1 = 3 + 1 = 4.$

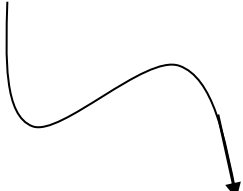
for $i = 4; i = 4 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
prev = temp;  
temp = temp->next;
```

prev = *temp* = 0x1000.

temp = *temp* $\rightarrow$ *next* = 0x2000.





	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
<i>* next</i>	0x3018	NULL

while(*Temp* = 0x2000 $\neq$ NULL) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

for *i* = 0; *i* = 0 < *temp* $\rightarrow$ *numElements* = 4 $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

if(*temp* $\rightarrow$ *Elements*[0]: 60 $\neq$ 110) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{False} \end{array} \right]$

i = *i* + 1 = 0 + 1 = 1.

for $i = 1; i = 1 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[1]: 70 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 1 + 1 = 2.$

for $i = 2; i = 2 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[2]: 80 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 2 + 1 = 3.$

for $i = 3; i = 3 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[3]: 90 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

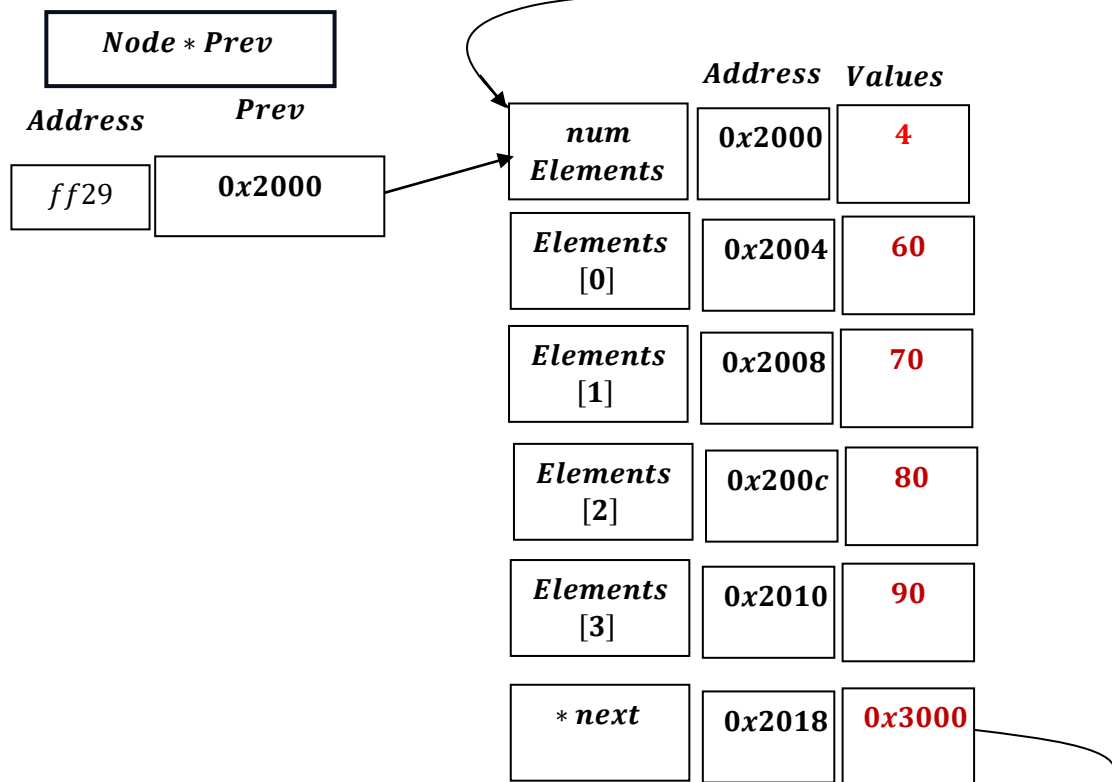
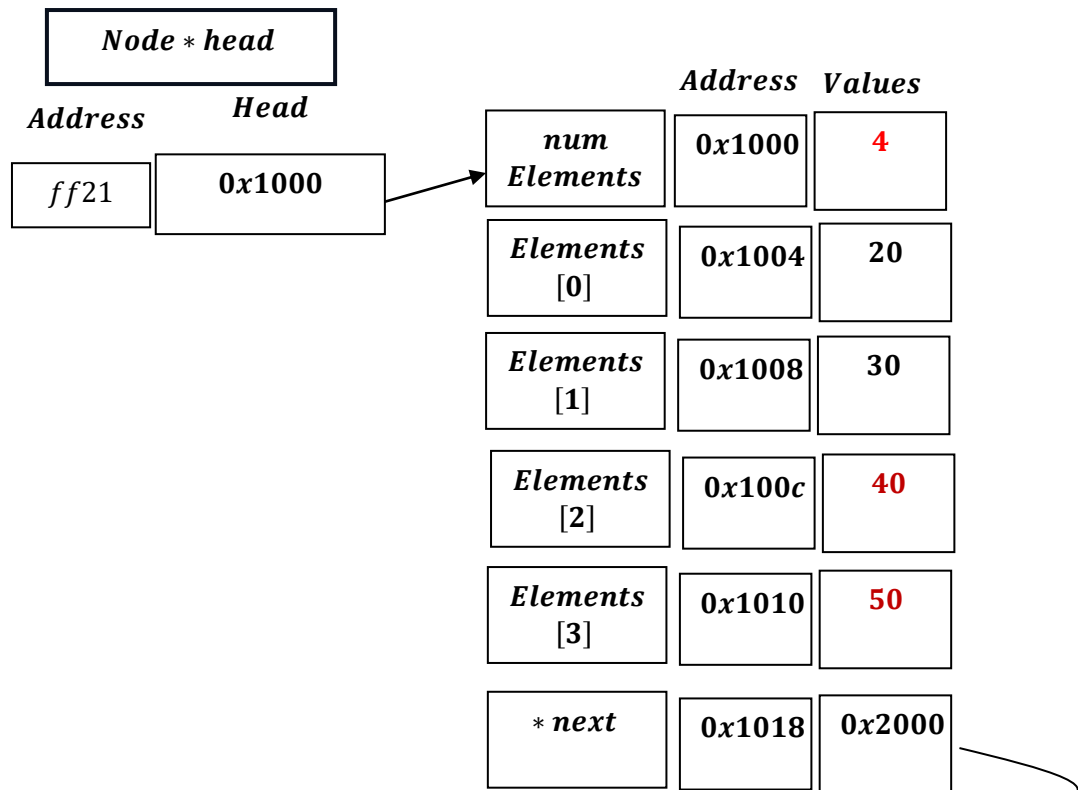
$i = i + 1 = 3 + 1 = 4.$

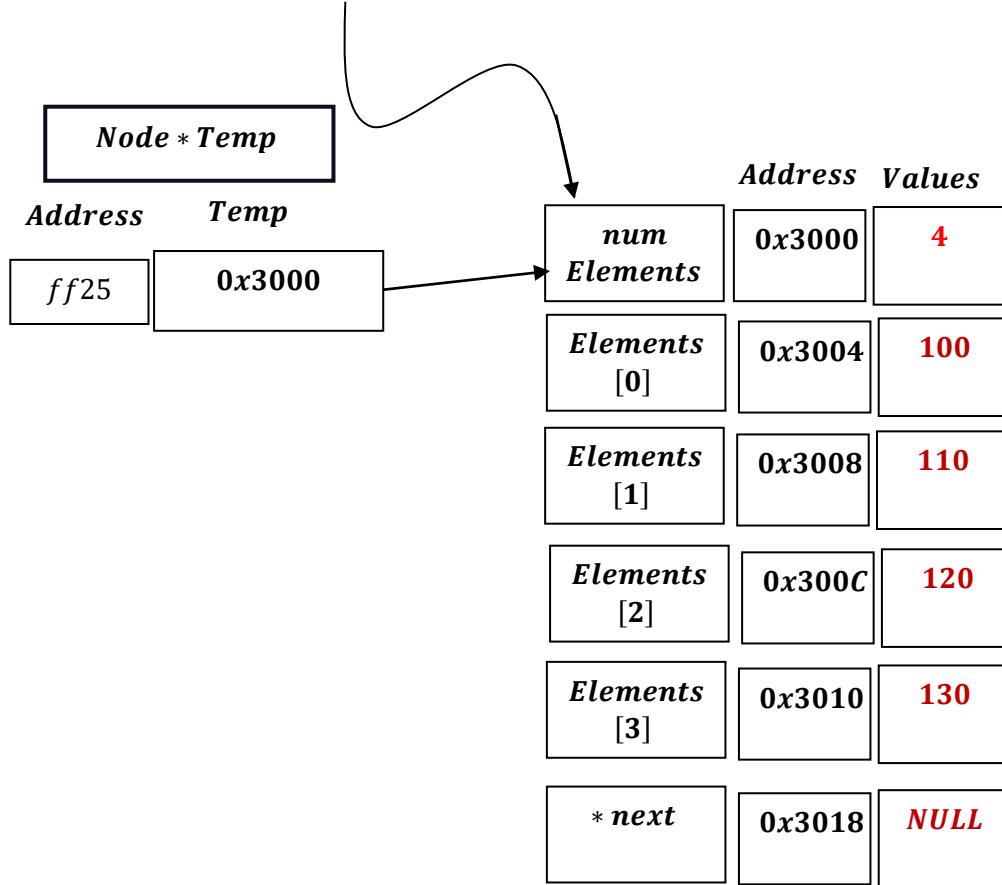
for $i = 4; i = 4 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
prev = temp;  
temp = temp->next;
```

prev = *temp* = 0x2000.

temp = *temp* $\rightarrow$ *next* = 0x3000.





while(Temp = 0x3000 $\neq$ NULL) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

for $i = 0; i = 0 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if(temp $\rightarrow$ Elements[0]: 100 $\neq$ 110) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 0 + 1 = 1.$

for $i = 1; i = 1 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if(temp $\rightarrow$ Elements[1]: 110 = 110) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

Shifting of Elements

```

for (int j = i; j < temp->numElements - 1; j++) {
    temp->elements[j] = temp->elements[j + 1];
}
temp->numElements--;

cout << "Element " << val << " deleted
successfully.\n";

```

$for : int j = i = 1 ; j < temp \rightarrow numElements - 1 [j < (4 - 1 = 3)] \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$temp \rightarrow element[j = 1] = temp \rightarrow element[j + 1 = 1 + 1 = 2] = 120$

$j = j + 1 = 1 + 1 = 2$

$for : int j = 2 ; j < temp \rightarrow numElements - 1 [j < (4 - 1 = 3)] \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$temp \rightarrow element[j = 2] = temp \rightarrow element[j + 1 = 2 + 1 = 3] = 130$

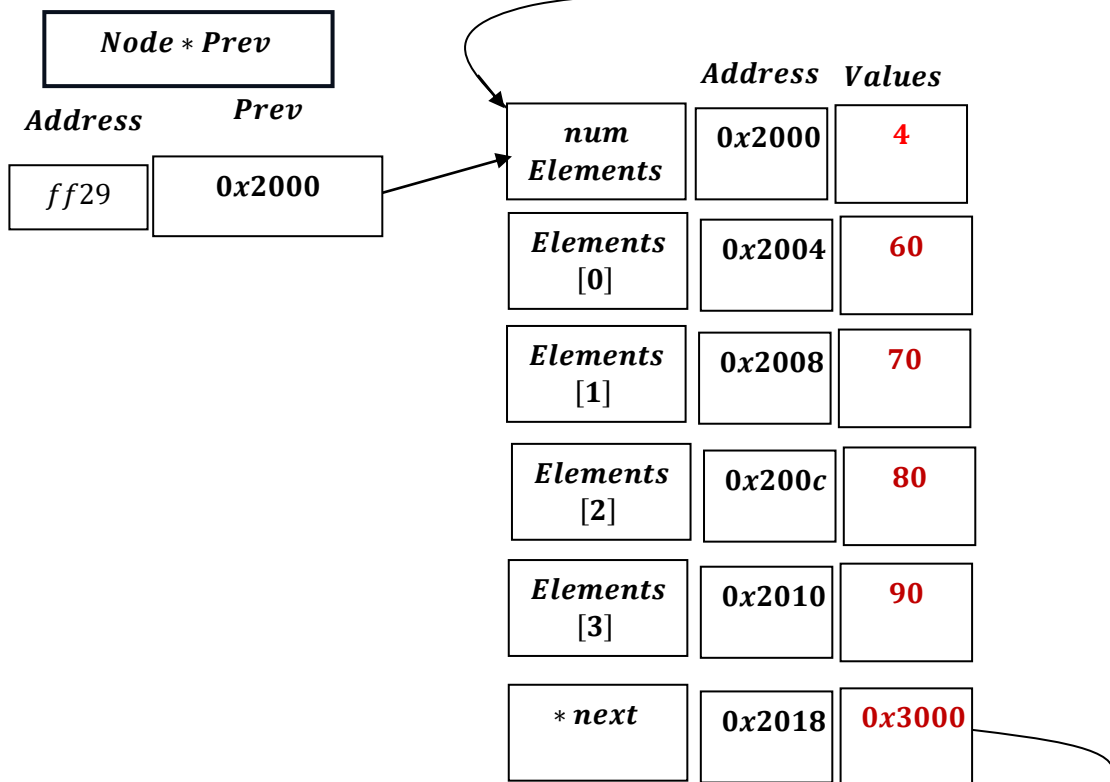
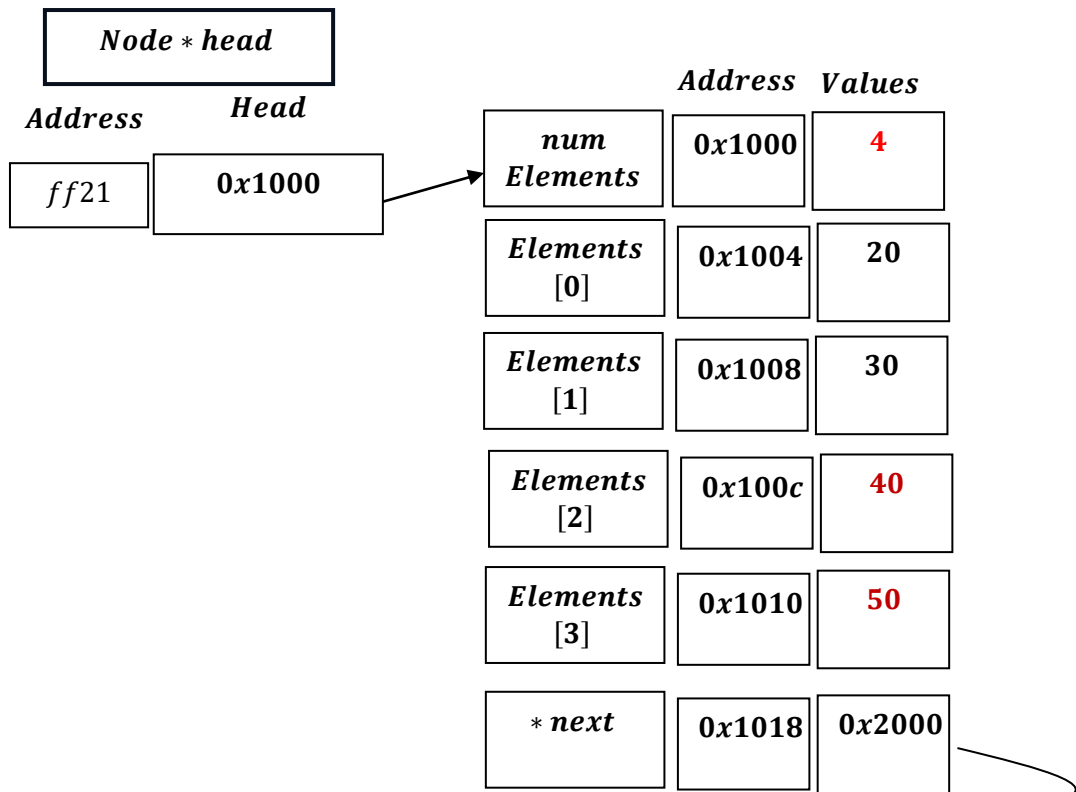
$j = j + 1 = 2 + 1 = 3$

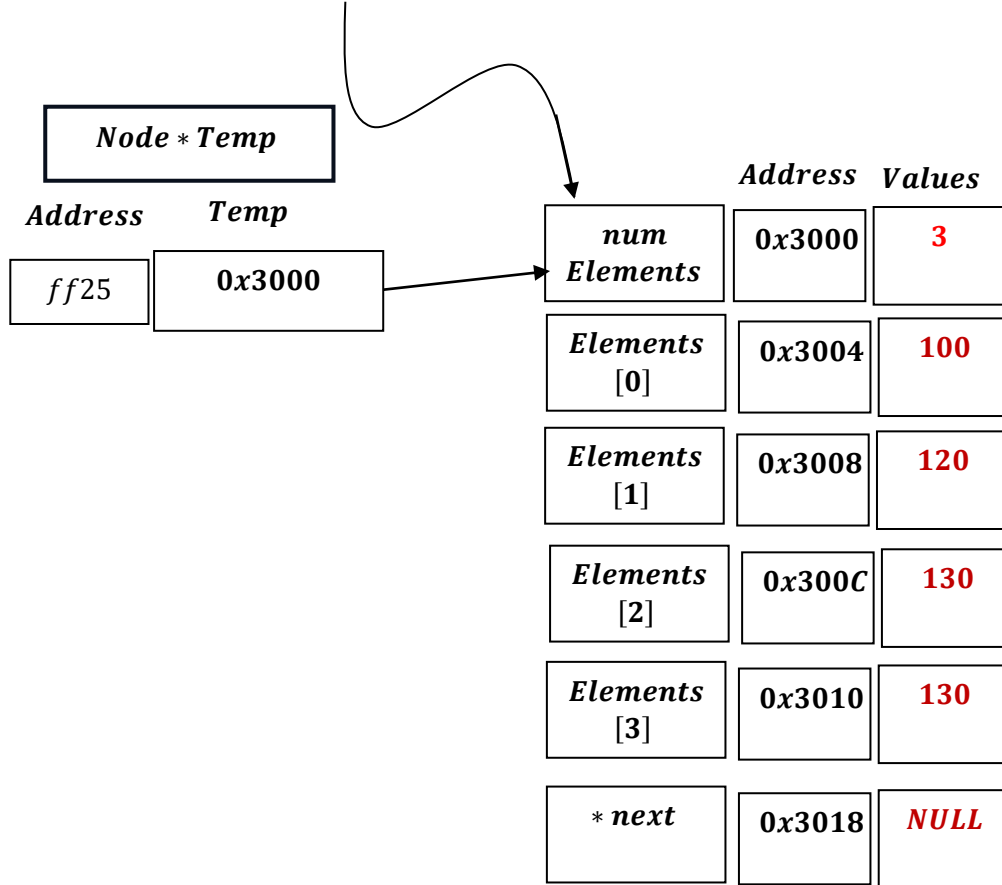
$for : int j = 3 ; j < temp \rightarrow numElements - 1 [j < (4 - 1 = 3)] \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

**Hence
loop
Exits**

$temp \rightarrow numElement = temp \rightarrow numElements - 1 = 4 - 1 = 3.$

Print: `` Element `` val: 110 ``deleted successfully.``





return;

Return , will return **void** [Return type void delete function.] and exit from the loop.

Note, Element[3] → from Address: 0x3000, contains 130 , but UpperBound i. e. numElements has become **3** , therefore next time temp , when it will be pointing at : 0x3000, for loop will run:

for $i = 0; i = 0 < temp \rightarrow numElements = 3$ [Condition is True]

i. e. **loop condition will be true when :**

$i = 0, i = 1, i = 2$. and will not reach : Elements[3].

Now, when : temp → numElements = 0?

A. List has a single Node ,B. When A node's numElements become 0.

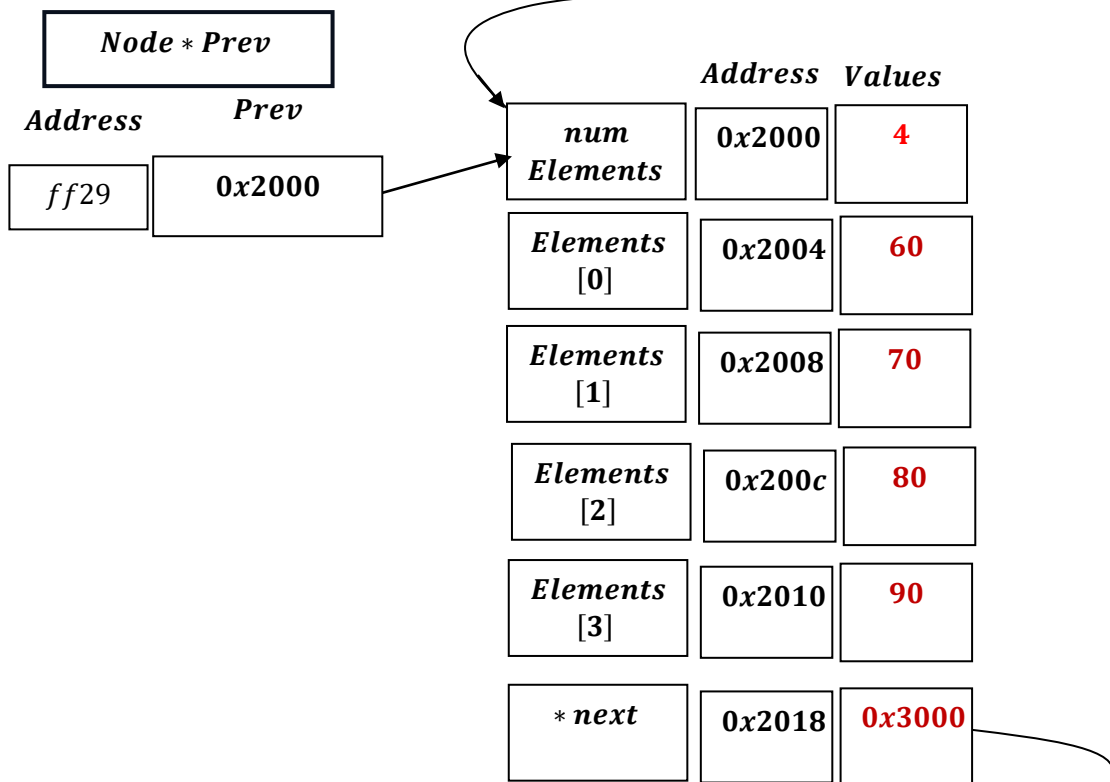
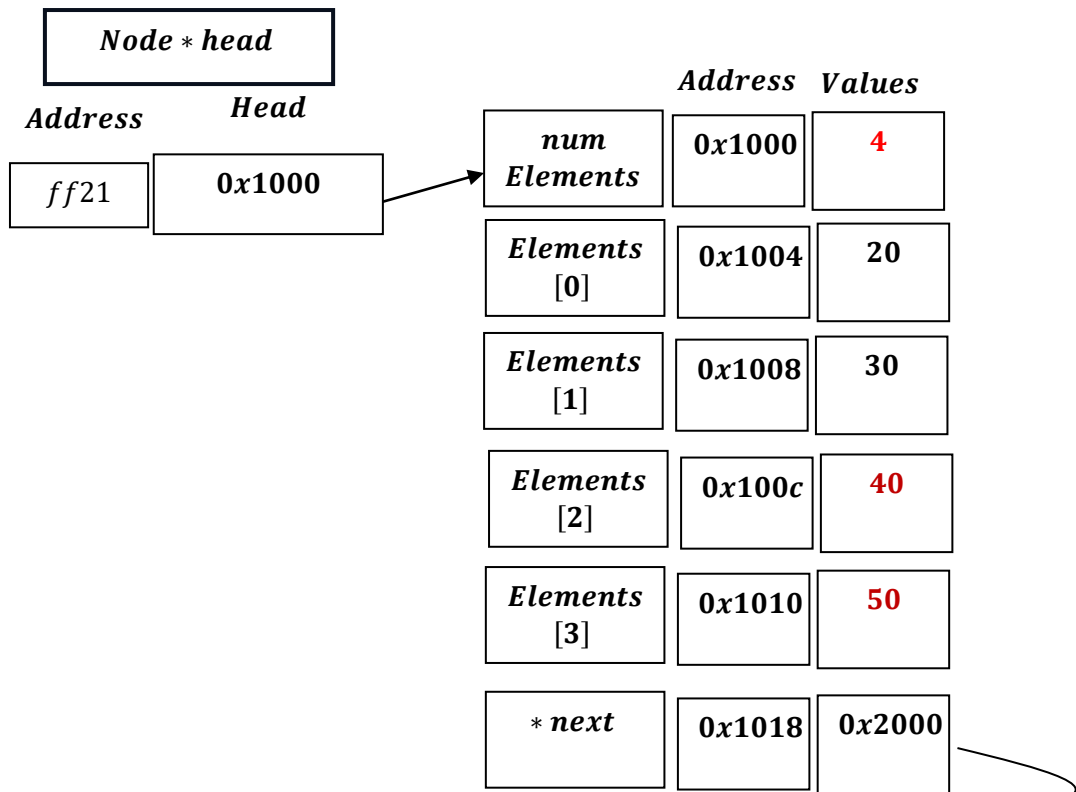
```
// Check if the node is now completely empty
if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {
        prev->next = temp->next;
        free(temp);
    }
}
cout << "An empty node was removed from the list.\n";
}
```

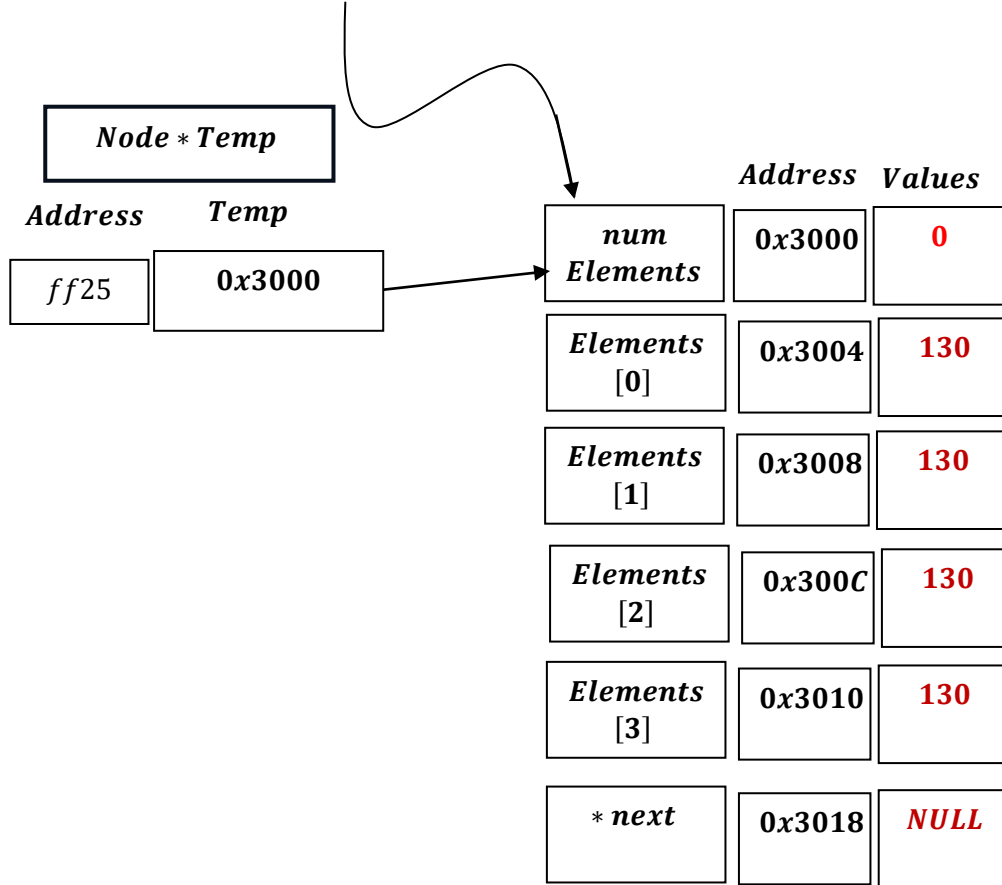
B. When a node's numElements become 0

i. e. no elements left in that node to get

Deleted.

Then : Prev ≠ NULL is True.





```

if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {
        prev->next = temp->next;
        free(temp);
    }
}
cout << "An empty node was removed from the list.\n";
}

```

$$if (temp \rightarrow numElements = 0 = 0) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$$

$$if(prev: 0x2000 = NULL) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$$

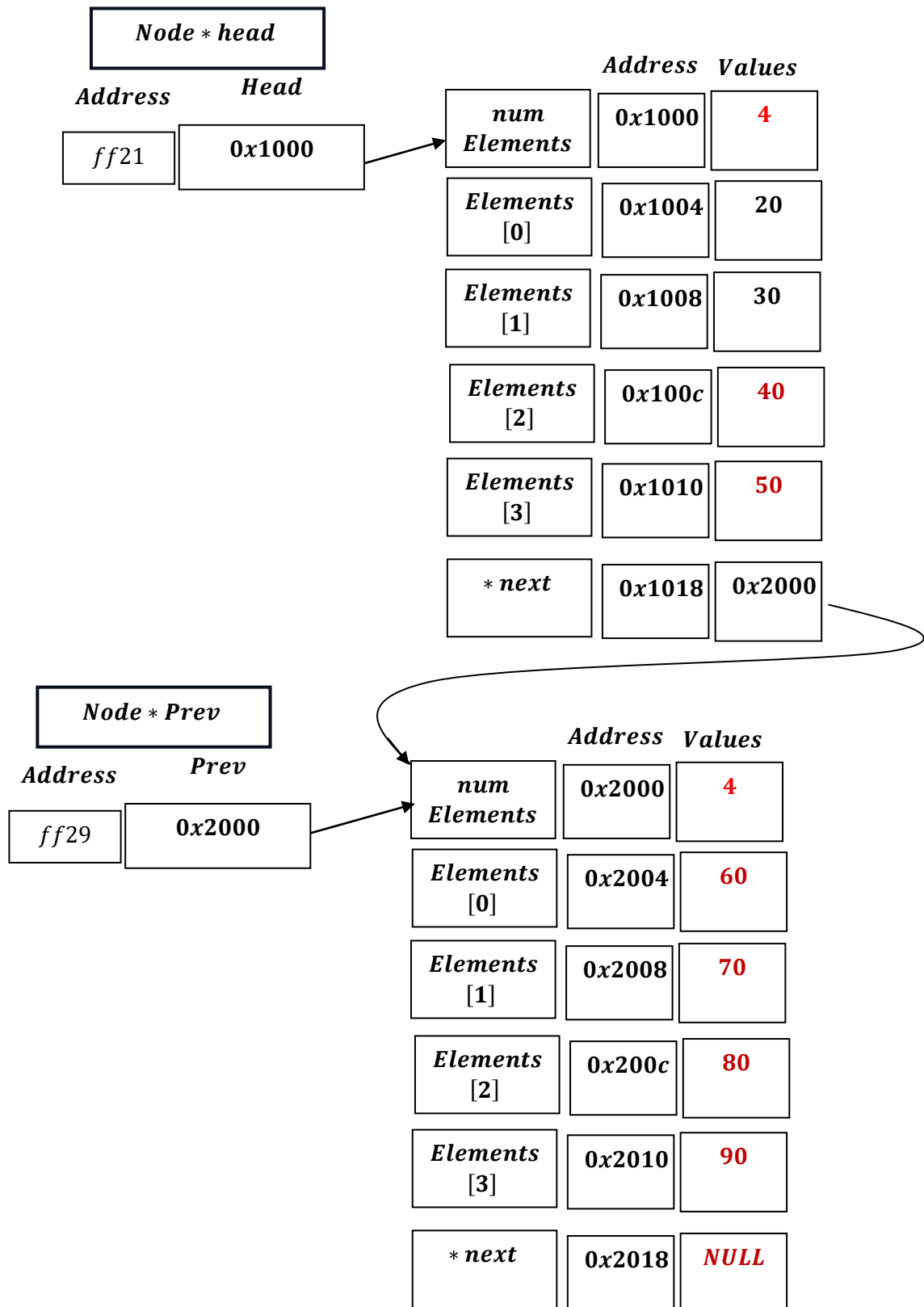
Hence ,Else part:

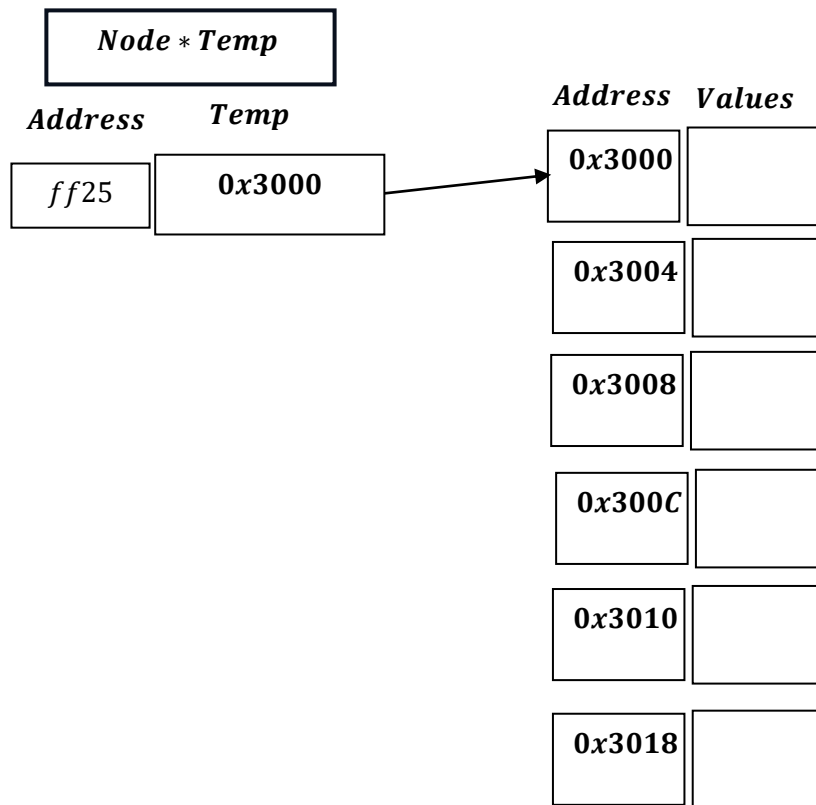
```
else {  
    prev->next = temp->next;  
    free(temp);  
}  
cout << "An empty node was removed from the list.\n";
```

prev: 0x2000 → next = temp: 0x3000 → next = NULL.

free(temp).

*Next it will return **void** Return type
void delete
function. and exit from the loop.*



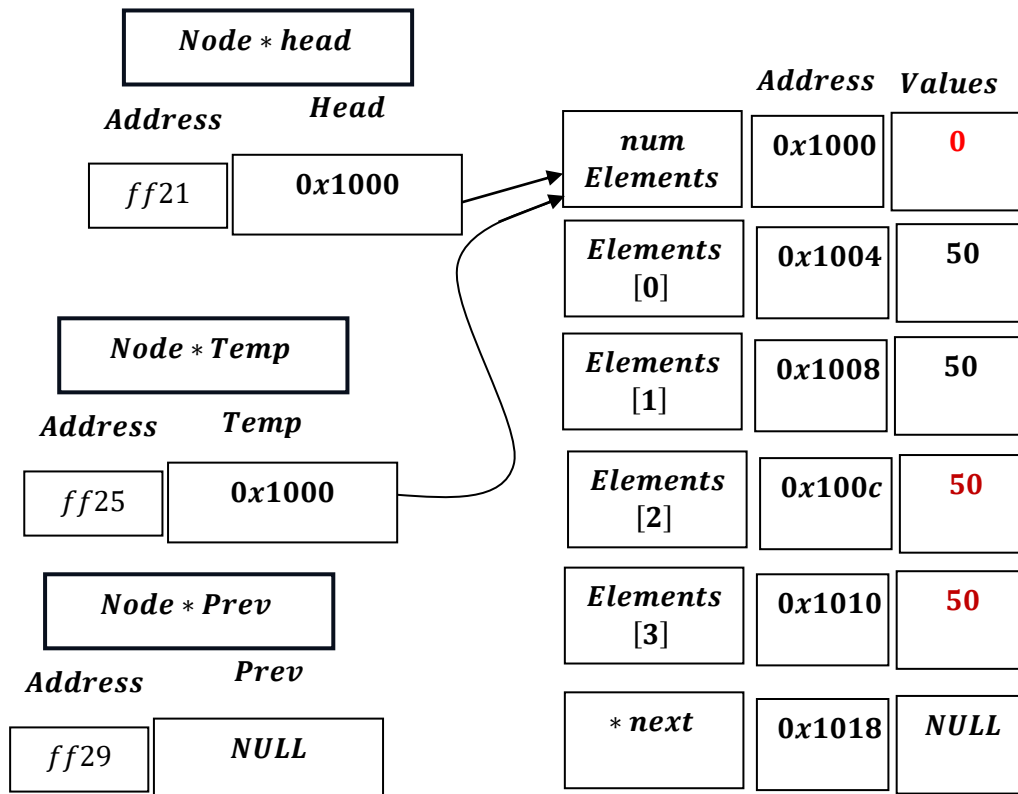


A. List has a single Node

and

temp → numElements = 0 i. e. No elements left in that single node to get deleted.

Then: Prev = NULL is True.



```

    if (temp->numElements == 0) {
        if (prev == NULL) {
            head = temp->next;
            free(temp);
        } else {...}
    }
    cout << "An empty node was removed from the list.\n";
}

```

Temp → *numElements* = 0 [True]

Prev = *NULL* [True]

head = *temp* → *next* = *NULL*

free(temp);

Next it will return *void* [Return type
void delete
function.] and exit from the loop.

